SAKE: Spatial Question Answering over Knowledge Graph based on Embedding Techniques

Huan Li[†] Hua Lu[‡] Lidan Shou[§] Ke Chen[§] Gang Chen[§]

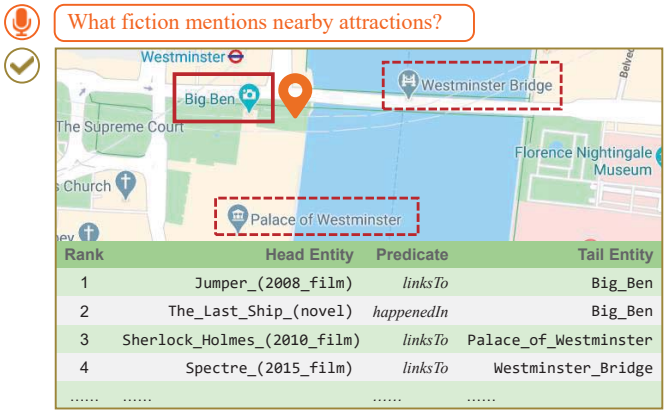
[†]Department of Computer Science, Aalborg University, Denmark
[‡]Department of People and Technology, Roskilde University, Denmark
[§]College of Computer Science and Technology, Zhejiang University, China
lihuan@cs.aau.dk, luhua@ruc.dk, {should, chenk, cg}@zju.edu.cn

***Abstract*—As knowledge graphs (KGs) are increasingly acquiring spatial aspects, natural language spatial question answering (NLSQA) in KG is pertinent for location-based applications. The major challenges for NLSQA include data heterogeneity due to spatial and non-spatial aspects in KG, recognition of spatial constraints from questions, and integration of spatial closeness and semantic relevance in answers. In this work, we propose a framework for Spatial question Answering over Knowledge graph based on Embedding techniques (SAKE). For entities and predicates in KG, SAKE captures their spatial implications and semantics in spatial fingerprints and semantic embeddings, respectively. On top of the embeddings, a question answering process is trained offline, which mainly learns a similarity function to measure the joint spatial and semantic similarity between question and answer. The learned similarity function is employed by a candidate search algorithm to retrieve the answers for each online question. We design comprehensive experiments with real datasets to evaluate SAKE. The evaluation results verify SAKE's effectiveness and efficiency.**

I. Introduction

Knowledge graphs (KGs) are widely used in natural language question answering applications [7], [12], [20]. With the ubiquity of location sensing, KGs nowadays are increasingly augmented with spatial data/knowledge. Taking Wikidata [29] as an example, by June 2018, 6.24M entities out of its 48M entities have spatial attributes such as GPS locations. When augmented with spatial awareness about the world, KGs can boost novel location-based applications [5], [18], [26].

One important type of application in KGs with spatial awareness is *natural language spatial question* (NLSQ) answering. An example is shown in Fig. 1. When Ann travels to Central London, she may ask "What fiction mentions nearby attractions?" to her mobile assistant (e.g., Siri or Google Assistant) to know interesting background knowledge about the place she is now visiting. The query information implied in this question, from a KG perspective, corresponds to some entity with type/property information ('fiction' and 'attraction') and predicate ('mention') in KG. Meanwhile, the respective answers can be given as a list of spatial facts, each being a triple (e_h, r, e_t) where e_h is a head entity, e_t is a tail entity, and r is a predicate that describes a relation of e_h and e_t.[1] In this example, Ann learns that film "Jumper (2018)" relates to her currently located attraction Big Ben by the fact[2] (`Jumper_(2008_film)`, *linksTo*, `Big_Ben`). Moreover, from the third and fourth returned facts, Ann also gets similar knowledge about other attractions spatially close to Big Ben. By mapping human language questions to triple-form results, NLSQ answering is able to provide easy-to-understand and spatial-aware results to end-users without any knowledge about query formulation.



Rank	Head Entity	Predicate	Tail Entity
1	Jumper_(2008_film)	*linksTo*	Big_Ben
2	The_Last_Ship_(novel)	*happenedIn*	Big_Ben
3	Sherlock_Holmes_(2010_film)	*linksTo*	Palace_of_Westminster
4	Spectre_(2015_film)	*linksTo*	Westminster_Bridge
.....	.....	.....	.....

Fig. 1: An Example of Spatial Question Answering

Answering NLSQs in KG is technically challenging. It is already well known that natural language question answering (NLQA) must cross *lexical gaps* to find relevant KG facts [12]. In addition, answering NLSQs in KG faces extra barriers:

Challenge 1 *(Data heterogeneity caused by the addition of spatial attributes).* NLSQs involve the *spatial query intent* that requires returning spatially close answers. However, spatial closeness is implicitly revealed by GPS locations while rarely captured as KG relations. It is impossible to meet spatial query intent by only extracting a spatial relation from NLSQ texts.

Challenge 2 *(Recognition of spatial constraints).* A user may specify a spatial constraint by giving phrases like 'my nearby place' (user's physical location must be considered) or location names like 'a Westminster road'. These constraints must be handled properly before searching for spatial facts.

Challenge 3 *(Integration of spatial closeness and semantic relevance).* It is important to embed spatial closeness into fact

[1]This paper studies the simple spatial question that targets a single KG triple as studies show simple questions generally meet common user needs [7], [16].

[2]All example data in this paper comes from YAGO [21] with a prefix URI http://yago-knowledge.org/resource/.

search. E.g., the answers in Fig. 1 consider Ann's nearest attraction Big Ben, and two others spatially close to her. This calls for effective integration of spatial closeness and semantic relevance as they lie in different dimensions in KG.

To address these challenges, we propose a framework for Spatial question Answering over Knowledge graph based on Embedding techniques (*SAKE*). The embedding techniques have shown advantages in handling lexical gaps and relation reasoning of NLQA [7], [16]. After representing questions and answers as latent vectors, a direct vector comparison can fill the lexical gap. Meanwhile, algebraic operations on vectors of entities and predicates enable us to synthesize facts unseen in KG. Beyond the idea of embedding techniques for NLQA, SAKE is able to model and extract spatial query intents from NLSQs, and bridge the spatial awareness gap between NLSQs and spatial facts. In particular, SAKE consists of an unsupervised process for embedding KG facts and a supervised process for spatial question answering (SQA).

In the first and fundamental part of SAKE, we obtain from the KG particular data views and translate the spatial and semantic information of KG entities and predicates to embeddings. On the one hand, we learn for each spatial entity a **spatial fingerprint** based on the absolute locations and spatial relationships of spatial entities. The spatial fingerprints capture spatial attributes as embeddings that allow matching to the query intents of NLSQs in the embedding space (for Challenge 1). On the other hand, we obtain standard KG embeddings to preserve KG relations and concatenate them with name and type information to generate the final semantic embeddings.

The SQA processing takes two phases. In the *offline* phase, the training pairs of NLSQ and answer are prepared, where each answer is represented by the spatial and semantic embedding outcomes of its entities and predicates. A neural network is trained to extract from an NLSQ the spatial query intent as a spatial fingerprint that can recall spatially close entities (for Challenge 2). Neural networks are also trained to map the NLSQ to the semantic embeddings of its mentioned entity and predicate. In the spaces of spatial fingerprints and semantic embeddings, a joint similarity function is learned to find answers that are spatially and semantically closest to an NLSQ (for Challenge 3). Through these efforts, we are able to map NLSQs to their intended answers in KG. In the *online* phase, for each incoming NLSQ, we extract its spatial fingerprint and semantic embedding, and pass them to a search algorithm that retrieves a list of facts closest to the NLSQ.

To sum up, this paper makes the following contributions.

- We define the NLSQ answering over KG and propose an embedding-based framework to resolve it. (Section II).
- We design SQA-oriented KG embedding techniques, particularly a spatial KG embedding method capturing the spatial relationships/properties of entities. (Section III).
- We devise a set of SQA processing techniques, namely the neural networks to extract spatial fingerprint and semantic embeddings from an NLSQ, a joint similarity function for ranking candidates, and an efficient algorithm to search for the best-matched facts. (Section IV).
- We conduct extensive experimental evaulations. The results verify SAKE's effectiveness and efficiency. (Section V)

In addition, Section VI reviews the related work; Section VII concludes the paper and discusses future work.

II. Preliminaries

Table I lists the notations used throughout this paper.

TABLE I: Notations

Symbol	Meaning
e_i, e_j^s	an entity, a spatial entity
E, E^s	entity set, spatial entity set
r_k, R	a predicate, predicate set
$t = (e_h, r, e_t)$	a KG fact
$\mathbf{e}_i$, $\mathbf{r}_k$	the full semantic embedding of e_i / r_k
e_i^{es} / e_j^{is}	an Explict/Implict Spatial Entity
$\mathbf{se}_j$	the spatial fingerprint of e_j^s
$\hat{\mathbf{se}}^q$	spatial query intent of an NLSQ q^s
$\Psi(t, q^s)$	similarity between fact t and NLSQ q^s

A. Problem Analysis and Formulation

A KG is a formal representation of facts describing the subjects and their relations that embody some or all domains of perception. Formally, let $E = \{e_1, \ldots, e_N\}$ be the set of **entities** denoting the domain subjects, L the set of **literals** referring to the values of entities (e.g., number, string, and date), and $R = \{r_1, \ldots, r_M\}$ the set of **relations** (or predicates) linking two entities or an entity and a literal. A **fact** in KG can be represented as a triple $t \in E \times R \times (E \cup L)$. A fact triple that does not pre-exist in KG is a *synthesized fact*.

We define the **class-entity** as a special type of entity that indicates the class of some entities. In KG, an entity's class is captured by a fact with a predicate *isA*, e.g., (`Big_Ben`, *isA*, `Attraction`) and (`Attraction`, *isA*, `Place`). Here, `Attraction` and `Place` are both class-entities.

Furthermore, for a **spatial entity**, at least one of its classes or literals is spatial. E.g., spatial classes can be `Country` and `Shop`, whereas spatial literals can be point coordinates, lines, polygons, etc. An entity is spatial if one of its superior class-entities in the class hierarchy is spatial. E.g., `Attraction` is a spatial entity as we know its class-entity `Place` is spatial. KG's class hierarchy is usually small so that it is easy to select and maintain a list of spatial class-entities. We use E^s to denote the set of spatial entities in KG.

Definition 1 (Spatial Fact)**.** *A triple $t = (e_h, r, e_t)$ is a spatial fact if its head entity or its tail entity is a spatial entity. Formally, all spatial facts form a set $T^s = \{(e_h, r, e_t) | e_h \in E^s\} \cup \{(e_h, r, e_t) | e_t \in E^s\}$.*

Definition 2 (Natural Language Spatial Question, NLSQ)**.** *An NLSQ q^s is a natural language question that can be answered by a list of spatial facts. Each of them interprets the answer by matching q^s's mentioned relation and class and being close to q^s's mentioned spatial range if applicable. Since it is impossible to exhaust all NLSQ patterns, this study selects four typical NLSQs that are identified based on how the entity, entity class, and spatial range of interests are uttered:*

1) *A* basic NLSQ *asks for some entities having the mentioned relation with the mentioned entity.*

2) *A* class-based NLSQ *asks for some mentioned class of entities having the mentioned relation with the mentioned entity or mentioned class of entities.*
3) *A* nearby NLSQ *extends a class-based NLSQ such that the asked entities should also be close to the user's location.*
4) *A* location-name NLSQ *extends a class-based NLSQ such that the asked entities should also be close to the location whose name is mentioned by the question.*

Example 1. *Q1-Q4 in Fig. 2 are the examples of the basic, class-based, nearby, and location-name NLSQs that interest the London traveler Ann, respectively. E.g., Q4 aims to find a pair of spatial entities* $(?e_h^s, ?e_t^s)$ *whose relation is specified as 'has' and classes are specified as 'museum' and 'road'. All their fact patterns involve at least one spatial entity (in red). A class can be specified for a non-spatial entity (Q2/Q3) and a spatial entity (Q3/Q4). Particularly, the spatial entities asked by Q3/Q4 are* location-dependent *as some phrase like 'nearby' or an explicit location name (e.g., 'Westminster') is used. That means when searching for possible spatial entities, we should extract underlying spatial query intent from the user location (Q3) or location name in NLSQ (Q4).*

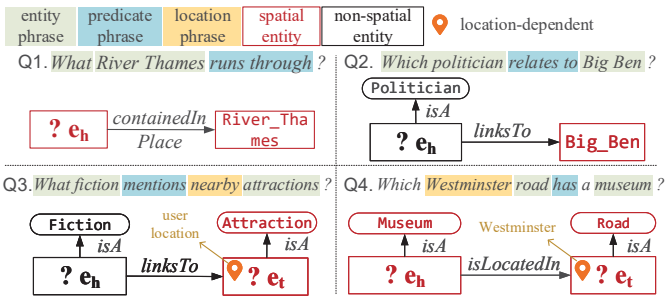


Fig. 2: Examples of Four Types of NLSQs to Study

NLSQs targeting literals can be simply addressed by general QA methods [16], [24]. Hence, NLSQs here only consider the relation of two entities, *not* an entity and a literal. NLSQs that involve more complex semantics, like "Find me *another* museum in Westminster?", are beyond the scope of the paper.

Research Problem (Spatial Question Answering over Knowledge Graph, KGSQA)**.** *Given a KG and an NLSQ* q^s*, KGSQA returns* k *spatial facts in a set* T_q^s *where* $T_q^s = \{(e_h, r, e_t) | e_h \in E^s\} \cup \{(e_h, r, e_t) | e_t \in E^s\}$, $|T_q^s| = k$, *and* $\forall t \in T_q^s, \Psi(t, q^s) \leq \Psi(t', q^s)$ *for any other fact* $t' \notin T_q^s$.

Above, $\Psi(t, q^s)$ captures the ranking score of a spatial fact t with respect to the NLSQ q^s. We consider the similarity between t and q^s based on three criteria.

Criterion 1. The spatial entities included in t must match the spatial query intent implied by q^s.

Criterion 2. The entities and predicate of t must be semantically close to their counterparts mentioned in q^s.

Criterion 3. If t is a synthesized fact, its predicted relation of the entities must be reliable and rational.

The specific design of $\Psi(t, q^s)$ will be given in Section IV-C.

To enable the similarity measurement between q^s and a candidate KG fact, we convert them into embeddings. The preliminaries are given below.

B. Embedding Techniques for KG

By representing facts (entities and predicates) as latent vectors with the graph structure and property information maximumly preserved, KG embedding techniques facilitate data mining tasks over KG. Classic distance-based KG embedding techniques [6], [14], [19] translate a fact (e_h, r, e_t) into its embedding form $(\mathbf{e}_h, \mathbf{r}, \mathbf{e}_t)$ as follows. First, vectors $\mathbf{e}_h$, $\mathbf{r}$, $\mathbf{e}_t$ are initialized with random filled-ins [6], [19] or values inherited from the word embedding of their corresponding entity/predicate [14]. Subsequently, a function $f(\cdot)$, typically $\mathbf{e}_t \approx f(\mathbf{e}_h, \mathbf{r})$, is defined to capture the relation r of e_h and e_t in embedding space. For example, TransE [6] uses $\mathbf{e}_t \approx \mathbf{e}_h + \mathbf{r}$ and TransR [19] uses $\mathrm{P}_r\mathbf{e}_t \approx \mathrm{P}_r\mathbf{e}_h + \mathbf{r}$ where P_r is a projection matrix of predicate r. Finally, the embedding model is trained by minimizing $||\mathbf{e}_t - f(\mathbf{e}_h, \mathbf{r})||_2$ over all sampled facts in KG.

KG embedding has been widely used in question answering (i.e., KGQA). Given a question q and a KG $\mathcal{G}$, the general idea is to project q and each possible fact t in $\mathcal{G}$ into some embedding space and measure their similarity to retrieve the closest fact to q [7]. Formally, the similarity function is defined as $S(q, t) = ||\mathbf{f}(q), \mathbf{g}(t)||$ where $||\cdot, \cdot||$ is a distance metric, and $\mathbf{f}, \mathbf{g}$ are two functions that embed q's and t's features as a k-dimension vector $\mathbb{R}^k$, respectively. The answer to q is then given as $\hat{t}(q) = \arg\min_{t' \in \mathcal{G}} S(q, t')$. A margin-based ranking loss $[\Delta - S(q_i, t_i) + S(q_i, t_i')]_+$ is used to learn function parameters, where $[\cdot]_+$ keeps the positive part of the formula, Δ is a predefined margin score, (q_i, t_i) is a question-answer pair (QA-pair) for training, and t_i' is the corresponding *corrupted* (i.e., negative sampled) fact. The question embedding function $\mathbf{f}$ can be designed based on characters [12], words [7], [16], or lexical features (n-grams) [32]. The fact embedding function $\mathbf{g}$ can be designed based on basic features (e.g., IDs, names of entities and/or predicates) [7], [12], logical features (e.g., question category, entity type, and logical predicate) [32], or paraphrasing features (e.g., answer path, answer types, and 1-hop entity/predicates) [10]. Existing embedding-based KGQA methods [7], [11], [12], [16], [32], however, do not consider the spatial query intent of an NLSQ; neither do they learn spatial similarity in embedding spaces. In contrast, our proposed KGSQA framework integrates spatial closeness and semantic relevance in an NLSQ.

C. SAKE Framework

Fig. 3 depicts the SAKE framework. It consists of SQA-oriented KG embedding (Section III) and SQA processing (Section IV). Given a KG, we translate its entities/predicates into spatial and semantic embeddings. As no ground truth is available, the embedding procedures are unsupervised, with the objective of preserving the inherent spatial or semantic relations/properties of entities. The outputs in turn serve as frozen vectors for training the SQA processing.

The KG embedding involves two independent processes: First, the spatial KG embedding is performed on a KG's spatial data view that only presents the spatial entities and their relations. In particular, a *spatial encoding* is employed to convert each spatial entity with an explicit geospatial location or

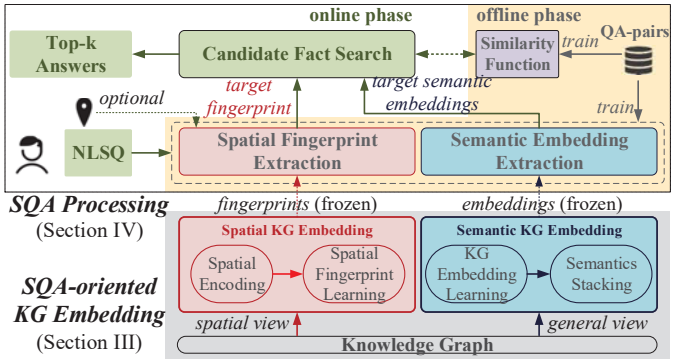


Fig. 3: Overall Framework of Embedding-based KGSQA

range as a location-specified vector. In contrast, spatial entities without geospatial attributes are encoded randomly. A *spatial fingerprint learning* is then conducted to update the initial encodings and return the ultimate fingerprints that capture the location properties and spatial relationships of spatial entities. Second, the semantic KG embedding is performed on KG's general view. It conducts a standard *KG embedding learning* to capture their KG relations. After that, a *semantics stacking* is employed to integrate the learned KG embeddings with pre-trained word embeddings for KG labels and entity classes.

The SQA processing is divided into offline and online phases. The offline phase builds a model to map NLSQs to their corresponding KG facts based on training data (QA-pairs). In this model, the *spatial fingerprint extraction* module captures the spatial query intent of an NLSQ q^s, i.e., the spatial range of the intended spatial fact. The *semantic embedding extraction* module maps q^s to its mentioned head entity[3] and predicate in semantic KG embedding space. Also, a *similarity function* $\Psi(t, q^s)$ is learned to measure a fact t's joint similarity to q^s in terms of q^s's spatial fingerprint and semantic embeddings. In the online phase, an incoming NLSQ is mapped to its target fingerprint (possibly mapped from the user location) and semantic embeddings, and a *candidate fact search* algorithm is called to rank the candidates based on $\Psi(t, q^s)$. The top-k closest facts in the rank are returned as the answers to the NLSQ. Analogous to browsing web search results, the user can select her/his intended answer(s) based on the similarity-based ranking and data displayed on the map.

III. SQA-ORIENTED KG EMBEDDING

A. Spatial KG Embedding

We generate an L^{se}-dimensional vector as the *spatial fingerprint* for each spatial entity to facilitate the selection of candidate spatial facts in SQA processing. We differentiate two types of spatial entities. **Explicit spatial entities** (ESEs) link to some geospatial locations or ranges in KG, but **implict spatial entities** (ISEs) do not. Provided that the spatial range of an NLSQ was recognized, we can locate the intended ESEs through spatial filtering over their geospatial attributes (e.g., using GeoSPARQL [28]). However, this method is insufficient to find the ISEs. To this end, we derive the pseudo spatial fingerprint of an ISE through its KG relations with other ESEs. The steps of spatial KG embedding are illustrated in Fig. 4.

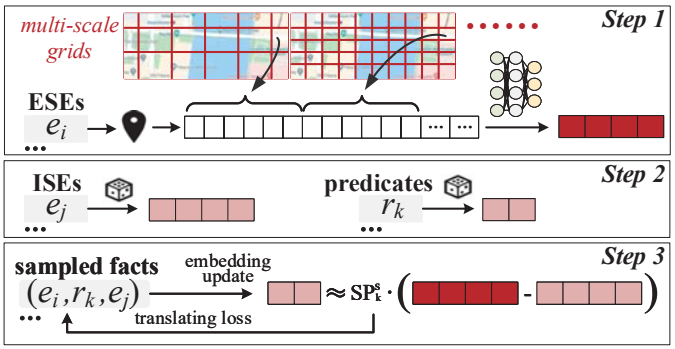


Fig. 4: Steps of Spatial KG Embedding

Step 1) Encoding ESEs. For each ESE $e_i^{\text{es}} \in E^{\text{es}}$, we obtain its geospatial attributes (e.g., GPS location or polygon) and create an initial fingerprint encoding $\mathbf{se}_i^{\text{es}} \in \mathbb{R}^{L^{\text{se}}}$. Encoding geospatial attributes is difficult because spatial entities have different spatial extents—it is not sufficient to encode ESEs all at the same scale. Therefore, we employ a multi-scale location encoding method [22]. We take an example of a GPS location $\mathbf{l} \in \mathbb{R}^2$ within a geospatial area $\mathcal{A}$. We use a set of grid scales $\{s_1, \ldots, s_G\}$ to divide $\mathcal{A}$ into uniform grids at different scales. At each scale s_z ($1 \leq z \leq G$), the location $\mathbf{l}$ is converted into a periodic representation of grid cells, i.e., $PR_z(\mathbf{l}) = [PR_i^1(\mathbf{l}); PR_i^2(\mathbf{l}); PR_i^3(\mathbf{l})]$ where

$$PR_z^j(\mathbf{l}) = [\cos(\frac{\mathbf{l}^\top \cdot \mathbf{a}_j}{s_1 \cdot (s_G/s_1)^{\frac{z-1}{G}}}); \sin(\frac{\mathbf{l}^\top \cdot \mathbf{a}_j}{s_1 \cdot (s_G/s_1)^{\frac{z-1}{G}}})] \quad (1)$$

and $\mathbf{a}_1 = [1, 0]$, $\mathbf{a}_2 = [-1/2, \sqrt{3}/2]$, $\mathbf{a}_3 = [-1/2, -\sqrt{3}/2]$ are three vectors with a 120 degree angle to each other. Each scale $PR_z^j(\mathbf{l})$ amounts to 6 dimensions and the total encoding size of $PR(\mathbf{l})$ is $6G$. The encoder $PR(\mathbf{l})$ captures a location $\mathbf{l}$'s grid features at multiple scales by a hexagon pattern. As features under different spatial density distributions are computed, the multi-scale encoding mitigates the periodicity issues of sine and cosine functions in Equation 1.

Alternative to a single point $\mathbf{l}$, an ESE e_i^{es} may be a geospatial range $a \subseteq \mathcal{A}$. We use an ESE's geospatial range (if no then GPS point) to generate its fingerprint, i.e.,

$$\mathbf{se}_i^{\text{es}} = \begin{cases} \text{NN}(PR(\mathbf{l}')), \mathbf{l}' \sim \mathcal{U}(e_i^{\text{es}}.a) & \text{if } e_i^{\text{es}}.a \neq \varnothing; \\ \text{NN}(PR(e_i^{\text{es}}.\mathbf{l})) & \text{otherwise.} \end{cases} \quad (2)$$

where $\mathbf{l}' \sim \mathcal{U}(e_i^{\text{es}}.a)$ is a sample location over the uniform distribution of the spatial range $e_i^{\text{es}}.a$. Since the encoding of spatial fingerprint will be performed multiple times during the training, the uniform sampling of $\mathbf{l}'$ will learn the properties of $e_i^{\text{es}}.a$ eventually. In particular, $\text{NN}(\cdot)$ is a neural network with ReLU layers to convert an original $6G$-dimensional representation to an L^{se}-dimensional spatial fingerprint. The use of $\text{NN}(\cdot)$ is to enable the intricate feature learning on the location encoding. We use a matrix $\mathbf{E}^{\text{es}} \in \mathbb{R}^{N^{\text{es}} \times L^{\text{se}}}$ to store

[3] An alternative solution is learning the tail entity. To ease the presentation, this paper only elaborates on the SQA processing based on the head entity.

all N^{es} ESEs' spatial fingerprints.

Step 2) Encoding ISEs and Predicates. For each ISE $e_j^{\text{is}} \in E^{\text{is}}$, we fill in random real values to create its pesudo spatial fingerprint $\mathbf{se}_j^{\text{is}} \in \mathbb{R}^{L^{\text{se}}}$ due to the lack of explicit geospatial information. As a result, we obtain an $N^{\text{is}} \times L^{\text{se}}$ fingerprint matrix $\mathbf{E}^{\text{is}}$ where N^{is} is the number of ISEs. Likewise, we randomly initilize an embedding matrix $\mathbf{R}^{\text{s}} \in \mathbb{R}^{M^{\text{s}} \times L^{\text{sr}}}$ where M^{s} is the number of predicates involved in all ESE/ISEs and L^{sr} is the predicate embedding size. For each such predicate embedding $\mathbf{sr}_k \in \mathbf{R}^{\text{s}}$, we initilize an $L^{\text{sr}} \times L^{\text{se}}$ projection matrix to conjugate those predicate embeddings and the spatial fingerprints. Each embedding $\mathbf{se}_j^{\text{is}}$ or $\mathbf{sr}_k$ is normalized and can be found from the corresponding matrix by index.

Step 3) Spatial Fingerprint Learning. We proceed to update the initial encodings to make them consistent with the relations in the spatial KG view. Formally, we denote each spatial fact as $t = (e_i, r_k, e_j), e_i, e_j \in E^s, E^s = E^{\text{es}} \cup E^{\text{is}}$. Following the idea of distance-based KG embedding (c.f. Section II-B), the loss of translating t to the embeddings is defined as

$$\Phi^s(e_i, r_k, e_j) = ||\mathbf{sr}_k - \text{SP}_k^r \cdot (\mathbf{se}_i - \mathbf{se}_j)||_2 \tag{3}$$

. To make optimal spatial fingerprints in $\mathbf{E}^{\text{es}}$ and $\mathbf{E}^{\text{is}}$, we expect such loss for the facts in the spatial view is minimized. We employ a margin-based ranking optimization method, aiming to expand the margin between the loss of the true facts and that of the corrupted (false) facts.

$$\min_{\theta_{\text{NN}}, \mathbf{E}^{\text{is}}, \mathbf{R}^{\text{s}}, \mathcal{SP}} \sum_{t \in T^s} \sum_{t^- \in \mathbb{N}(t)} [\Delta - \Phi^s(t) + \Phi^s(t^-)]_+ \tag{4}$$

where T^s is a set of facts sampled from the spatial KG view, θ_{NN} parameterizes the network $\text{NN}(\cdot)$ in Equation 2, $\mathcal{SP} = \{\text{SP}_1^r, \ldots, \text{SP}_{M^{\text{s}}}^r\}$ is the set of projection matrices corresponding to all M^{s} predicates, Δ is a predefined margin score, and $[\cdot]_+$ attains non-negative values only. Besides, $\mathbb{N}(t)$ is a set of negative samples of t as

$$\begin{aligned}\mathbb{N}(t) = &\{(e_i', r_k, e_j) | e_i' \in E^s \wedge e_i' \neq e_i\} \\ &\cup \{(e_i, r_k, e_j') | e_j' \in E^s \wedge e_j' \neq e_j\}\end{aligned}$$

This negative sampling is to corrupt the head or tail entity in t with a randomly selected spatial entity.

Equation 4 is optimized by stochastic gradient descent in minibatch mode [19]. For each epoch, we sample a new set T^s of facts and their corresponding sets of negative facts to compute the gradients for updating θ_{NN}, $\mathbf{E}^{\text{is}}$, $\mathbf{R}^{\text{s}}$, and $\mathcal{SP}$. We add a constraint that the $\ell 2$-norm of all embeddings is 1, lest the training process trivially minimizes the objective by increasing embedding norms [6]. The training process converges when the objective over a set of validation facts is less than a given stopping threshold.

After the training, each ESE, GPS location $\mathbf{l}$ or geospatial range a, can be translated to a spatial fingerprint using Equation 2. Moreover, each ISE can find its pseudo fingerprint in $\mathbf{E}^{\text{is}}$. We do not generate fingerprints for non-spatial entities as they usually link to multiple spatially distributed entities.

B. Semantic KG Embedding

Likewise, two KG embedding matrices $\mathbf{E}^\star \in \mathbb{R}^{N \times L^{\text{e}}}$ and $\mathbf{R}^\star \in \mathbb{R}^{M \times L^{\text{r}}}$ are initialized from uniform distributions. N and M (L^{e} and L^{r}) refer to the number (embedding size).

KG Embedding Learning. Similar to learning spatial fingerprints, margin-based ranking loss (see Equation 4) is applied to optimize $\mathbf{E}^\star$ and $\mathbf{R}^\star$, in which the translating loss of a sampled fact $t(e_i, r_k, e_j)$ is defined as $\Phi(t) = ||\mathbf{r}_k^\star - \text{P}_k^r \cdot (\mathbf{e}_i^\star - \mathbf{e}_j^\star)||_2$ where $\mathbf{e}_i^\star, \mathbf{e}_j^\star \in \mathbf{E}^\star$, $\mathbf{r}_k^\star \in \mathbf{R}^\star$ are KG embeddings, and $\text{P}_k^r \in \mathbb{R}^{L^{\text{r}} \times L^{\text{e}}}$ is the projection matrix for predicate r_k. Different from the negative sampling in spatial fingerprint learning, we replace the head or tail entity with an entity having the same category (i.e., spatial or non-spatial) with it.

The above embeddings only capture KG relations, while not corresponding to the name and class information that appears in an NLSQ. Unlike existing techniques [16], [20] that use fuzzy matching to find candidates online, we perform a semantics stacking to append name and class features into pre-trained KG embeddings before the SQA.

Semantics Stacking. We capture the semantic meaning of each entity by embedding the words of its labels and classes in KG. The advantage is that word embedding brings two semantically similar words close to each other in the vector space. E.g., given an entity `Big_Ben`, we obtain its KG labels as ['Big Ben', 'Elizabeth Tower', ...]. We map each word in these labels as a vector and average them to form the word embedding of `Big_Ben`'s labels. The mapping is based on GloVe model [25] in which word embeddings have been pre-trained using large web corpora. The same method is applied to generating word embedding of `Big_Ben`'s class names as ['Bell Tower', 'Towers in England', ...]. For each predicate, we obtain the word embedding of its relation name only. We stack the learned KG embedding and word embedding(s) to form a final semantic embedding $\mathbf{e}$ or $\mathbf{r}$ for each entity or predicate. Let the GloVe embedding size be V, we have the final embedding size $L^{\text{e}} + 2V$ for $\mathbf{e}$ and $L^{\text{r}} + V$ for $\mathbf{r}$. The matrices for stacked semantic embeddings are denoted as $\mathbf{E}$ and $\mathbf{R}$, respectively. We demonstrate the effect of using semantics stacking in Section V-D.

IV. SQA Processing

We train the SQA processing with a set of QA-pairs, each being an NLSQ with entity/predicate mentions and a fact represented as frozen embeddings obtained in Section III.

A. Spatial Fingerprint Extraction

We represent each NLSQ q^s as a sequence of tokens $W_q = \{w_1, \ldots, w_Q\}$ and extract the spatial fingerprint from W_q in two cases. First, we process W_q to determine if the intended answer is relevant to the current user location $\mathbf{l}_q$. If so, we directly convert $\mathbf{l}_q$ into a spatial fingerprint based on Equation 2. Otherwise, we identify the relevant location name (e.g., 'Westminster' or 'Big Ben') from W_q as a semantic embedding $\mathbf{e}^q$ of size $L^{\text{e}} + 2V$. Since a spatial entity has both semantic embedding and spatial fingerprint, we find a spatial

entity having the closest semantic embedding with $\mathbf{e}^q$, and use its fingerprint to represent the spatial query intent of q^s.

To this end, we design a neural network architecture as illustrated in Fig. 5. The model takes W_q as input, and outputs a flag $y \in \{0, 1\}$ that indicates if $\mathbf{l}_q$ should be converted to fingerprint and a semantic embedding $\mathbf{e}^q$ that represents the location name mentioned in W_q. The labels y and $\mathbf{e}^q$ are obtained while generating NLSQs (see Section V-A).

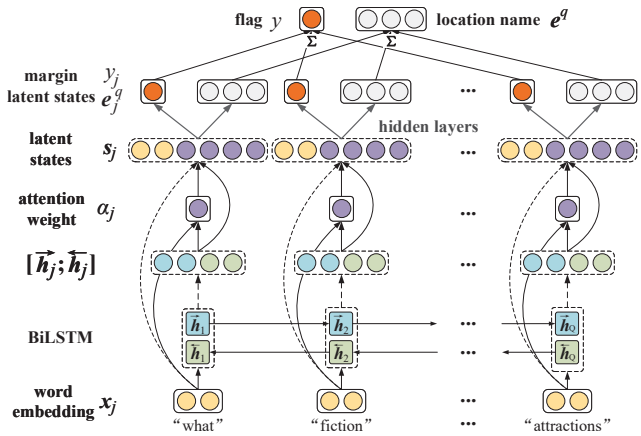


Fig. 5: Spatial Query Intent Learning

Given $W_q = \{w_1, \ldots, w_Q\}$, we convert each word token to its GloVe word embedding $\mathbf{x}_j$. Subsequently, a BiLSTM architecture is employed to learn the forward and backward hidden state sequences $(\overrightarrow{\mathbf{h}}_1, \ldots, \overrightarrow{\mathbf{h}}_Q)$ and $(\overleftarrow{\mathbf{h}}_1, \ldots, \overleftarrow{\mathbf{h}}_Q)$. On top of LSTM, we concatenate the forward and backward hidden state vectors and obtain $\mathbf{h}_j = [\overrightarrow{\mathbf{h}}_j; \overleftarrow{\mathbf{h}}_j]$. Then, we calculate each token's attention weight α_j as follows.

$$\alpha_j = \frac{\exp(\phi_j)}{\sum_{i=1}^{Q} \exp(\phi_i)}; \quad \phi_j = \tanh(\mathbf{w}_\phi^\mathsf{T}[\mathbf{x}_j; \mathbf{h}_j] + b_\phi)$$

The attention weight α_j is applied to $\mathbf{h}_j$ and consequently we obtain the concatenated vector as $\mathbf{s}_j = [\mathbf{x}_j; \alpha_j \mathbf{h}_j]$.

For each $\mathbf{s}_j$, we add several hidden layers to generate a scalar state y_j, which indicates if the corresponding token $\mathbf{x}_j$ carries the meaning like 'nearby' or 'this place'. Consequently, the ultimate flag y is computed as the mean of all tokens' scalar states, i.e., $y = 1/Q \sum_{j=1}^{Q} y_j$.

If the predicted value of y indicates that the user location $\mathbf{l}_q$ is relevant to spatial query intents, we directly use the multi-scale encoding of $\mathbf{l}_q$ as the spatial fingerprint. Otherwise, the spatial query intent should be specified by some particular location name in W_q. In this case, we reuse the latent state $\mathbf{s}_j$ to map it to the embedding of such a location name. Referring to Fig. 5, another hidden layer stack is applied to $\mathbf{s}_j$ to generate the margin latent state $\mathbf{e}_j^q \in \mathbb{R}^{L^\mathrm{e}}$ for each token $\mathbf{x}_j$. As a result, the location name of the overall sequence is obtained as $\mathbf{e}^q = 1/Q \sum_{j=1}^{Q} \mathbf{e}_j^q$.

Given an online NLSQ q^s, we learn its target semantic embedding $\hat{\mathbf{e}}^q$ from the trained neural network. Then, we search the matrix $\mathbf{E}$ for a spatial entity $e' \in E^s$ that has the closest semantic embedding with $\hat{\mathbf{e}}^q$. Subsequently, we search the matrix $\mathbf{E}^\mathrm{is}$ or $\mathbf{E}^\mathrm{es}$ for e''s fingerprint $\hat{\mathbf{se}}^q$ which in turn is used as q^s's spatial intent.

B. Semantic Embedding Extraction

We train a model to extract semantic embeddings of the head entity $\mathbf{e}_h$ and predicate $\mathbf{r}_o$ from W_q. The extracted embeddings enable us to find semantically closest facts. Our intuition of learning such extraction is that the target entity/predicate's information (e.g., name/alias, contextual entities, and entity type) has been captured in both W_q and the output embedding.

An architecture similar to Fig. 5 is shown in Fig. 6, which maps W_q to the semantic embedding of either the head entity or predicate. For brevity, we omit the BiLSTM layers between word embedding $\mathbf{x}_j$ and latent state $\mathbf{s}_j$. We construct a fully connected layer upon each $\mathbf{s}_j$ to generate the margin latent state $\mathbf{e}_j$ (or $\mathbf{r}_j$). The output of the model is the mean states, i.e., $\mathbf{e} = 1/Q \sum_{j=1}^{Q} \mathbf{e}_j$ (or $\mathbf{r} = 1/Q \sum_{j=1}^{Q} \mathbf{r}_j$).

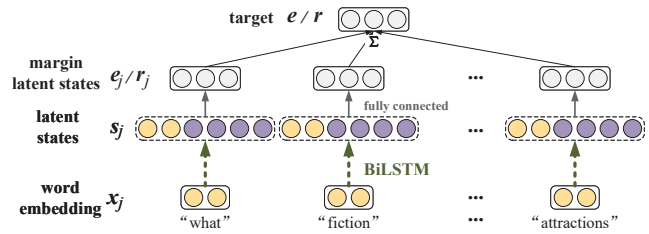


Fig. 6: Semantic Embedding Learning

Once the two models for entities and predicates are trained separately, we generate for an NLSQ q^s the target embeddings of its head entity and predicate, denoted as $\hat{\mathbf{e}}_h$ and $\hat{\mathbf{r}}_o$.

C. Joint Similarity Function for Candidate Fact Ranking

We define three categories of spatial facts as candidates.

1) **Tail Spatial Fact (TSF)** (e_h^n, r_o, e_t^s) where the head entity $e_h^n \notin E^s$ is a non-spatial entity and therefore the tail entity must be a spatial entity, i.e., $e_t^s \in E^s$.
2) **Head Spatial Fact (HSF)** (e_h^s, r_o, e_t^n) where the head entity $e_h^s \in E^s$ is a spatial entity but the tail entity $e_t^n \notin E^s$ is a non-spatial entity.
3) **Full Spatial Fact (FSF)** (e_h^s, r_o^s, e_t^s) where both entities $e_h^s, e_t^s \in E^s$ are spatial entities. In this case, the predicate r_o^s must have an embedding $\mathbf{sr}_o$ in $\mathbf{R}^s$ (see Section III-A).

Given a candidate fact $t = (e_h, r_o, e_t)$ and embeddings $\hat{\mathbf{se}}^q$, $\hat{\mathbf{e}}_h$ and $\hat{\mathbf{r}}_o$ extracted from an NLSQ q^s, we design a *joint* similarity function $\Psi(t, q^s)$ that combines the spatial closeness and semantic relevance as follows.

$$\Psi(t, q^s) = \Psi^\mathrm{sm}(t, q^s) + \Psi^\mathrm{sp}(t, q^s) \tag{5}$$

$$\begin{aligned}\Psi^\mathrm{sm}(t, q^s) = \lambda_\mathrm{h}||\mathbf{e}_h - \hat{\mathbf{e}}_h||_2 + \lambda_\mathrm{r}||\mathbf{r}_o - \hat{\mathbf{r}}_o||_2 \\ + \lambda_\mathrm{t}||\mathbf{r}_o^\star - \mathrm{P}_o^r(\mathbf{e}_h^\star - \mathbf{e}_t^\star)||_2\end{aligned}$$

$$\begin{aligned}\Psi^\mathrm{sp}(t, q^s) = \lambda_\mathrm{sr} \min(\mathrm{I}(e_t), \mathrm{I}(e_h))||\mathbf{sr}_o - \mathrm{SP}_o^r(\mathbf{se}_h - \mathbf{se}_t)||_2 \\ + \lambda_\mathrm{se} \min(\mathrm{I}'(e_h)||\mathbf{se}_h - \hat{\mathbf{se}}^q||_2, \mathrm{I}'(e_t)||\mathbf{se}_t - \hat{\mathbf{se}}^q||_2)\end{aligned}$$

where Ψ^sm and Ψ^sp refer to the semantic and spatial part of similarity scores, respectively, $\mathrm{I}(e_h)$ is an indicator that returns 1 if e_h is a spatial entity and 0 otherwise, $\mathrm{I}'(e_h)$ is another indicator that returns 1 if e_h is a spatial entity and ∞ otherwise, and all λ_* are tradeoff parameters. Note that KG

embeddings $\mathbf{e}^\star$ and $\mathbf{r}^\star$ (cf. Section III-B) cut off the GloVe embedding parts from their full semantic embeddings as GloVe part takes no effect in predicting the tail entity. Equation 5 unifies similarity scoring for different spatial fact types.

In $\Psi^{\text{sm}}(t, q^s)$, the first (second) term measures the semantic embedding similarity between the target head entity (predicate) and its candidate. The third term measures the relation reliability in terms of the embedding translating loss of candidate fact t. The first two terms and the last term adhere to Criteria 2 and 3 in Section II-A, respectively. $\Psi^{\text{sm}}(t, q^s)$ does not explicitly compare the semantic embedding of the tail entity because the KG relation formed by the head entity and predicate basically determines the tail entity.

In $\Psi^{\text{sp}}(t, q^s)$, the first term measures the translating loss in spatial fingerprint space for t (adhere to Criterion 3). This term takes effect only when both entities are spatial (controlled by $\min(\mathrm{I}(e_t), \mathrm{I}(e_h))$). The second term measures t's consistency with the spatial query intent $\hat{\mathbf{se}}^q$ (see Criterion 1 in Section II-A). Since we do not know if the head or tail entity should satisfy this condition, we differentiate two cases. If it is an FSF, the second term outputs the minimum of the two fingerprint distances computed for the head and tail entities. If it is a TSF/HSF, the second term uses the fingerprint distance of the only spatial entity (controlled by the two I' indicators).

A grid search over $(\lambda_{\text{t}}, \lambda_{\text{r}}, \lambda_{\text{h}}, \lambda_{\text{se}}, \lambda_{\text{sr}})$ is used to learn the optimal parameter combination. The following objective function is employed to expand the margin between the scores of the closest fact and those of other candidate facts.

$$\min_{\lambda_{\text{t}}, \lambda_{\text{r}}, \lambda_{\text{h}}, \lambda_{\text{se}}, \lambda_{\text{sr}}} \sum_{q^s \in \mathcal{Q}} [\Delta' - \Psi(t_g, q^s) + \frac{1}{K} \sum_{t' \in \mathcal{C}^K} \Psi(t', q^s)]_+ \quad (6)$$

where Δ' is a predefined margin score, $\mathcal{Q}$ is an NLSQ set, t_g is the closest fact of q^s, and $\mathcal{C}^K$ is a set of K candidate facts close to q^s excluding t_g.

D. Candidate Fact Search Algorithm

It is possible to explore each category defined above to find candidates. While this is time-consuming and involves excessive candidates in a mismatched category. To this end, we propose Algorithm 1 that expands all categories in one procedure and returns a best-matched category of candidates based on an incremental mechanism. To facilitate nearest neighbor search, we use two K-D Trees KD^{sm} and KD^{sp} to index semantic embeddings in $\mathbf{E}$ and spatial fingerprints in $\mathbf{E}^{\text{is}} \cup \mathbf{E}^{\text{es}}$. For each spatial entity $e_i^s \in E^s$, we obtain its spatial fingerprint $\mathbf{se}_i$ and semantic embedding $\mathbf{e}_i$, and generate $\mathbf{ce}_i = [\mathbf{se}_i; \mathbf{e}_i^\star]$ to predict the tail in the case of FSF. We index all $\mathbf{ce}_i$s with another K-D Tree KD^{c}.

The algorithm maintains a flag set $\mathcal{F}$ to indicate which categories are currently possible, three candidate sets for each category, and two sorted maps for candidate spatial entities and predicates (lines 1–2). After that, it selects k candidate spatial entities with the nearest spatial fingerprint to the input $\hat{\mathbf{se}}^q$ (lines 3–4). As the relationship between spatial entities and facts is one-to-many, the top-k spatial entities should incur at least k candidate facts for further processing. Next, the

Algorithm 1 SEARCH (spatial fingerprint $\hat{\mathbf{se}}^q$, semantic embeddings $\hat{\mathbf{e}}_h$, $\hat{\mathbf{r}}_o$, thresholds $\eta^e, \eta^r, \eta^{\text{c}}, k'$)

1: category flag set $\mathcal{F} \leftarrow \{\texttt{TSF}, \texttt{HSF}, \texttt{FSF}\}$
2: initilize candidate sets $\mathcal{C}_{\text{TSF}}$, $\mathcal{C}_{\text{HSF}}$, $\mathcal{C}_{\text{FSF}}$ and sorted maps $\mathcal{M}_{\text{se}}$, $\mathcal{M}_{\text{r}}$
3: *CSE* $\leftarrow$ find kNN spatial entities of $\hat{\mathbf{se}}^q$ on KD^{sp}
4: put $\langle e_x, ||\mathbf{se}_x, \hat{\mathbf{se}}^q||_2 \rangle$ in $\mathcal{M}_{\text{se}}$ for each $e_x \in$ *CSE*
5: *CR* $\leftarrow \{r_j \mid ||\mathbf{r}_j, \hat{\mathbf{r}}_o||_2 \leq \eta^r\}$
6: put $\langle r_j, ||\mathbf{r}_j, \hat{\mathbf{r}}_o||_2 \rangle$ in $\mathcal{M}_{\text{r}}$ for each $r_j \in$ *CR*
7: **for** each $\langle e_x, d_x \rangle \in \mathcal{M}_{\text{se}}$ **do**
8: **if** $||\mathbf{e}_x, \hat{\mathbf{e}}_h||_2 \leq \eta^e$ **then**
9: **for** each $\langle r_j, d_j \rangle \in \mathcal{M}_{\text{r}}$ **do**
10: **if** r_j has spatial embedding $\mathbf{sr}_j$ **then**
11: $\mathbf{ce}_t \leftarrow \mathbf{ce}_x - diag(\mathrm{SP}_j^{-1}, \mathrm{P}_j^{-1})[\mathbf{sr}_j; \mathbf{r}_j^\star]$
12: *ET* $\leftarrow \{e_y \mid ||\mathbf{ce}_y, \mathbf{ce}_t||_2 \leq \eta^{\text{c}}\}$ on KD^{c}
13: *filter*(*ET*, $\mathcal{C}_{\text{FSF}}$, $\texttt{FSF}$, $\mathcal{F}$)
14: **else**
15: $\mathbf{e}_t^\star \leftarrow \mathbf{e}_x^\star - \mathrm{P}_j^{-1}\mathbf{r}_j^\star$
16: *ET* $\leftarrow \{e_y \mid ||\mathbf{e}_y^\star, \mathbf{e}_t^\star||_2 \leq \eta^e\}$ on KD^{sm}
17: *filter*(*ET*, $\mathcal{C}_{\text{HSF}}$, $\texttt{HSF}$, $\mathcal{F}$)
18: **else**
19: **for** each $\langle r_j, d_j \rangle \in \mathcal{M}_{\text{r}}$ **do**
20: **if** r_j has spatial embedding $\mathbf{sr}_j$ **then**
21: $\mathbf{ce}_h \leftarrow \mathbf{ce}_x + diag(\mathrm{SP}_j^{-1}, \mathrm{P}_j^{-1})[\mathbf{sr}_j; \mathbf{r}_j^\star]$
22: *EH* $\leftarrow \{e_y \mid ||\mathbf{ce}_y, \mathbf{ce}_h||_2 \leq \eta^{\text{c}}\}$ on KD^{c}
23: *filter*(*EH*, $\mathcal{C}_{\text{FSF}}$, $\texttt{FSF}$, $\mathcal{F}$)
24: **else**
25: $\mathbf{e}_h^\star \leftarrow \mathbf{e}_x^\star + \mathrm{P}_j^{-1}\mathbf{r}_j^\star$
26: *EH* $\leftarrow \{e_y \mid ||\mathbf{e}_y^\star, \mathbf{e}_h^\star||_2 \leq \eta^e\}$ on KD^{sm}
27: *filter*(*EH*, $\mathcal{C}_{\text{TSF}}$, $\texttt{TSF}$, $\mathcal{F}$)
28: **if** $|\mathcal{F}| = 1$ and $|\mathcal{C}_{\mathcal{F}[0]}| \geq k$ **then break**
29: **return** top-k facts in a candidate set with the lowest overall scores
30: **function** *filter*(E', $\mathcal{C}'$, *flag*, $\mathcal{F}$)
31: **for** each $e' \in E'$ **do**
32: **if** E' is ET **then** add (e_x, r_j, e') to $\mathcal{C}'$
33: **else** add (e', r_j, e_x) to $\mathcal{C}'$
34: **if** $\mathcal{F}$.*has*(*flag*) and $|\mathcal{C}'| > k'$ **then**
35: compute each fact's similarity score by Equation 5
36: $\mathcal{C}'.\Psi_{k'} \leftarrow$ the sum of top-k' lowest scores
37: **if** $\mathcal{C}'.\Psi_{k'} > \Psi_{k'}$ of other category **then** $\mathcal{F} \leftarrow \mathcal{F} \setminus$ *flag*

algorithm selects the predicates whose semantic embedding's distance to $\hat{\mathbf{r}}_o$ is within the threshold η^r (lines 5–6).

Then, the algorithm iterates on $\mathcal{M}_{\text{se}}$ in ascending order of distance (lines 7–28). The three categories are differentiated according to whether e_x is spatial (line 8) and whether r_j connects two spatial entities (lines 10 and 20). As a result, FSFs, HSFs, and TSFs are handled respectively at lines 11–13/21–23, 15–17, and 25–27. In particular, the target embedding is predicted based on combined embeddings for FSFs or semantic embeddings for HSF/TSFs, and then used to expand candidate entities via corresponding K-D Tree (bounded by a distance threshold). For each expansion, a function *filter*$(\cdot)$ is applied (lines 30-37). First, the candidate facts are synthesized and added to the corresponding fact set $\mathcal{C}'$ (lines 32-33). Then, the function checks if the current category can be pruned (lines 34–37). It computes the similarity score of each fact in $\mathcal{C}'$ based on Equation 5 (line 35) and sums up the top-k' ($k' \ll k$ is a top sampling parameter) lowest scores (line 36). If the summed score is higher than that of another category, the current category is pruned as its results are less promising (line 37). After the expansion for each category, the algorithm determines if the search can stop. If only one category remains

and it includes more than k candidates, the algorithm exits the loop (line 28). Finally, the algorithm returns the top-k facts from one category with the lowest overall scores (line 29).

Error Analysis. Let $\mathcal{C}_1$ be the best-matched category found by the top-k' search and a_1 the average score of the top-k' facts. The scores of the $(k'+1)$-th to k-th facts in $\mathcal{C}_1$ must be within $[a_1, 1]$ and the expected average score of $\mathcal{C}_1$'s top-k facts is within $[a_1, a_1 + (k-k')/k]$. Hence, finding the best-matched category with a smaller k' introduces a larger error. Let $a_2 \geq a_1$ be the average top-k' score of any of the other two categories, $a_1 + (k-k')/k < a_2$ ensures that the top-k' search finds the category consistent with the top-k search. However, as $\Psi^{\text{sp}}(t, q^s)$ in Equation 5 penalizes the wrong entity type and usually makes $a_2 \gg a_1$, a very small k' is practically applicable. We experimentally verify this in Section V-G.

V. Experimental Studies

A. Datasets and Question Generation

We generate QA-pairs from YAGO3 [21], a spatially enhanced KG, following the workflow in Fig. 7.

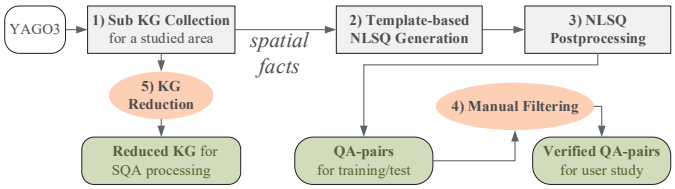


Fig. 7: Dataset Generation Workflow

1) Sub KG Collection. We select three populated cities as our study areas, namely London (LON), Copenhagen (CPH), and Beijing (BEJ). We create for each city the sub KG as follows.

1) We find all entities whose GPS are within the city's *minimum bounding rectangle* (MBR), and regard those having node degrees greater than 5 as the seeds. This removes some entities of low importance.
2) We obtain each seed entity's 1- and 2-hop relation facts to form a sub KG, because most of those beyond 2 hops are irrelevant to the studied city as we observed.
3) We remove a non-spatial entity if its node degree is less than 10 or greater than 300. Usually, these entities are either negligible or used to describe metadata with no actual meaning. Note that we only remove its relations irrelevant to other entities in the sub KG.
4) We query OpenStreetMap API [1] to obtain geospatial polygons for entities with GPS locations and around 0.3% entities have been augmented with a polygon. We then remove the entities out of the city's MBR. To test SAKE's ability to handle sparse geospatial attributes, we sample 10% (*default*), 20%, or 40% of them as ISEs by removing their geospatial attributes. We regard the rest as ESEs.

Datasets	LON	CPH	BEJ
# of spatial entities	13.6k	1.5k	7.3k
# of all entities	1.9M	0.3M	0.4M
# of predicates	71	73	76
# of FSFs	66.2k	6.7k	4.1k
# of HSFs	531.2k	442.3k	485.7k
# of TSFs	546.7k	33.1k	45.1k
# of all facts	17.6M	1.4M	2.1M

The statistics of the sub KGs are listed above.

2) Template-based NLSQ Generation. As large-scale QA-pairs are hard to obtain, we synthesize QA-pairs to train and evaluate our QA model following [7]. In particular, we feed each spatial fact to the following templates for basic NLSQs.

Fact	Condition of `r`	Basic NLSQ Template	
(?, r, e)	verb	*what* `r` *e?*	
	noun	*what is the* `r` *of* `e`*?*	*what is* `e`*'s* `r`*?*
(e, r, ?)	verb	*what does* `e` `r`*?*	*what is* `r`*-ed by* `e`*?*

The entity represented by `?` is the answer subject. We then generate other types of NLSQs (cf. Definition 2):

- Class-based NLSQs: We change 'what' in each basic template to 'which `?`.*type*' where `?`.*type* is one of the type name of entity `?`.
- Nearby NLSQs: We change 'what' to 'which [nearby | neighboring |...] `?`.*type*', and optionally change `e` to `e`.*type* in each basic template. For each such NLSQ, a user location is generated randomly within 1km from spatial entity `?`'s GPS location. This ensures that the meaning of 'nearby' is correctly reflected in QA-pairs.
- Location-name NLSQs: We change 'what' to 'which `?`.*loc_name* `?`.*type*', and optionally change `e` to `e`.*type* in each basic template. Specifically, a spatial entity `?`'s *loc_name* is obtained as `?`'s village or county-level district name using OpenStreetMap's reverse geocoding API [1].

Nearby and location-name NLSQs are only generated if `?` is a spatial entity. When filling in templates, we use entity and predicate aliases to replace them (1/3 probability). Aliases for entities are obtained as their label literals, while those for only a small set of predicates are manually maintained. For each spatial fact, we fill in all possible templates and sample a certain number of NLSQs to form QA-pairs. We filter out all facts related to the city entities (i.e., `London`, `Copenhegen`, and `Beijing`). The statistics of template-generated QA-pairs are summarized as follows.

Datasets	LON	CPH	BEJ
# of spatial facts	516.9k	35.7k	35.2k
# of basic NLSQs	186.3k	21.5k	15.5k
# of class-based NLSQs	1.03M	71.4k	70.3k
# of nearby NLSQs	969.9k	45.8k	32.7k
# of location-name NLSQs	688.6k	29.6k	20.2k
# of QA-pairs	2.87M	168.3k	138.7k

3) NLSQ Postprocessing. To improve the model generalization ability, we randomly augment 30% of the generated questions by grammatical transformations, e.g., changing 'which people' to 'who', 'which area' to 'where', 'is' to 'should be', and using case conversions.

4) Manual Filtering. We obtain subset datasets with human questions for the model verification in the following two ways:

Rephrasing. We sample 200 spatial facts' corresponding QA-pairs per dataset and ask 5 volunteers to mark out the QA-pairs whose fact they deem can answer the NLSQ. A pair is included if 3 out of 5 volunteers mark out it. After that, each included NLSQ is rephrased to be natural, by a volunteer with over 15 years of English learning experience (see samples in Table IV). As a result, we got 1161, 1029, and 1005 QA-pairs in the rephrased datasets named LON^R, CPH^R, and BEJ^R.

Mapping. 30MQA corpus [2] contains 30M human questions paired with the corresponding Freebase facts. We utilize

YAGO-Freebase mapping to find a 30MQA subset for our YAGO facts. In particular, for a spatial fact in our dataset, if it finds a Freebase fact whose head, relation, and tail are all matched, that Freebase fact's human question is mapped. For each dataset, we manually extend 20% mapped questions to nearby NLSQs (see NLSQ generation above) as 30MQA does not contain this type. We got 9150, 115, and 266 QA-pairs in the final mapped datasets LON^M, CPH^M, and BEJ^M.

5) KG Reduction. To verify the model's ability to synthesize facts, we randomly pick up 5% of facts from the test QA-pairs and remove them from the generated sub KG. That means some test questions' answers are unavailable in the reduced KG. We use reduced KGs to conduct SQA.

All datasets for empirical studies have been published[4].

B. Baselines and Performance Metrics

We consider five KGQA methods as summarized in Table II. Each method generally consists of three phases: 1) extracting entity and predicate information from the NLQ, 2) selecting candidate entities and then facts based on the extracted results, and 3) ranking facts based on the designed similarity function.

Each method treats spatial and non-spatial facts equally in all phases. To incorporate our spatial embedding technique into each method, we merge the spatial entities found in line 3 of Algorithm 1 into the candidate entities found in Phase 2) and employ the following similarity function in Phase 3):

$$\Psi(t, q^s) = (1 - \lambda^{\text{sp}}) \cdot \Psi^{\text{o}}(t, q^s) + \lambda^{\text{sp}} \cdot \Psi^{\text{sp}}(t, q^s) \quad (7)$$

where $\Psi^{\text{o}}(t, q^s)$ is the original method's normalized similarity score, $\Psi^{\text{sp}}(t, q^s)$ is the spatial similarity borrowed from Equation 5, and λ^{sp} is a tradeoff parameter that will be tuned optimally by grid search. This function rewards those answers having their spatial fingerprints close to q^s's spatial query intent. We suffix each such extended method with +S.

Two SAKE variants are designed: SAKE\GloVe skips the semantics stacking in the embedding extraction and similarity search; SAKE\S treats all spatial aspects as semantics and uses semantic embedding extraction in Phase 1) and $\Psi^{\text{sm}}(t, q^s)$ only as the similarity function in Phase 3).

For the end-to-end evaluation, we measure the recall at k ($R@k$), i.e., whether the closest fact appears in the top-k results returned for an NLSQ.

C. Implementation and Parameter Setting

The SAKE framework is implemented in Python 3 + PyTorch 1.4 + nltk 3.3, and run with an NVIDIA DGX-2 GPU in an AI cloud platform. The KG embedding module is extended from OpenKE [13]. We tried different parameter settings and report the best one in the experiments as follows. For spatial KG embedding, we tune $L^{\text{se}} = 64$, $L^{\text{sr}} = 64$, scale number $G = 16$ with minimum grid size 100m, negative sample set size $|\mathbb{N}| = 500$, and $\Delta = 1.0$. Three stacked hidden layers of size 256 are added to $\text{NN}(\cdot)$ in Equation 2. For semantic KG embedding, we tune $L^{\text{e}} = 100$, $L^{\text{r}} = 50$, $|\mathbb{N}| = 200$, and $\Delta = 0.6$. We load the GloVe word embeddings of size $V = 50$. We train the two embedding models using AdaGrad [16] with a learning rate $1e^{-4}$ and a batch size 100. A two-layer BiLSTM model with hidden state size 100 serves as the base of fingerprint extraction and semantic embedding extraction. Two hidden layers of size 300 are added between the latent state layer $\mathbf{s}_j$ and margin layer in Fig. 5. The dropout is set to 0.3 and Adam optimizer with learning rate $2e^{-4}$ is employed. For Equation 6, we tune $K = 10$, $\Delta' = 0.15$ and find the optimal normalized combination of $(\lambda_{\text{t}}, \lambda_{\text{r}}, \lambda_{\text{h}}, \lambda_{\text{se}}, \lambda_{\text{sr}})$ by grid search per dataset. For Algorithm 1, we tune $k' = 0.3k$, $\eta^e = 0.2$, $\eta^r = 0.01$, and $\eta^{\text{c}} = 0.3$. We use 8:1:1 training:validation:test split for the KG facts in KG embedding and the full QA-pairs in SQA processing.

We list key model parameters tuned for the baselines. In CQAA, the question LSTM-encoder and entity/predicate CNN-encoder are in a size of 300 and their decoders have output sizes of 100. The hidden state sizes are set to 300 for LSTMs/GRUs in BuboQA and UBSQA. The KG embedding dimension is set to 64 and two hidden state layers of size 300 are used for embedding learning in KGEQA. A bert-base-uncased pretrained model [3] is funetuned in BERTQA. Up to $5k$ matched entities are used to expand candidate facts in Phase 2) for all baselines.

D. Overall Comparison of KGSQA Performance

The comparisons in terms of $R@k$ on full test datasets, rephrased datasets, and Freebase-mapped datasets are reported in Table III. The following insights are derived.

First, similarity enabled by spatial fingerprints is crucial in SQA processing. For all datasets, all baselines' measures in the upper bar are clearly lower than their counterparts with fact search enhanced by spatial similarity. For example in LON, BuboQA+S increases $R@1$ by 32.3% on BuboQA and SAKE's $R@1$ is 36.4% higher than SAKE\S. The correct answers, especially to the nearby and location-name NLSQs, are greatly determined by spatial query intents. Such intents substantially perplex conventional KGQA methods (including SAKE\S) that lack the spatial fingerprint extraction. The study on different NLSQ types will be given in Section V-E.

Second, semantics stacking is necessary. This is evidenced by that SAKE always outperforms SAKE\GloVe. In Section III-B, the standard KG embeddings only capture certain KG relations of entities, carrying no meaning of entity names and classes from a textual perspective. In contrast, SAKE's semantics stacking makes a good supplement and fills the lexical gap between NLSQs and KG facts.

Third, SAKE has considerable advantages over all alternatives in synthesizing facts and in making use of joint similarity. CQAA+S perform the worst in all measures. It uses end-to-end training to map questions to entities/predicates directly, without capturing relations and textual information kept in KG. In contrast, BuboQA+S, UBSQA+S, and BERTQA+S all use fuzzy string matching to expand candidate facts. However, neither direct mapping nor fuzzy matching can recall those facts that are not in KG (recall that we have removed 5%

[4]https://github.com/sake-project/sake-preprocessing

TABLE II: Summary of Methodology of KBQA Baselines (see [2] for more details)

Method	information extraction from the NLQ in Phase 1)	candidate entity selection in Phase 2)	similarity function in Phase 3)
CQAA [12]	character-level attention LSTM to encode/decode questions coordinated with entity/predicate's CNN encodings	match n-gram of question to entity aliases to find candidate entities	product of likelihoods of entity and predicate
BuboQA [23]	BiLSTM and BiGRU to extract entity mention and classify predicate, respectively	fuzzy string matching for linking entity mentions to candidate KG entities	product of prediction scores of entity and predicate
UBSQA [24]	BiLSTM+CRF and BiLSTM to map to entity aliases and classify predicate, respectively	top candidate KG entities are decoded by Viterbi algorithm over CRF	product of likelihoods of entity and predicate
KGEQA [16]	BiLSTMs to map question to KG embeddings and entity mention, respectively	fuzzy string matching for linking entity mentions to candidate KG entities	combination of string similarity, prediction scores, and fact's link prediction score
BERTQA [20]	BERT to extract entity mention and predicate in one pass	*same as above*	combination of string similarity and prediction scores

TABLE III: Performance Comparison of SQA Methods (best results are in bold)

Dataset	LON	CPH	BEJ	LON^R		CPH^R		BEJ^R		LON^M		CPH^M		BEJ^M	
Metric (unit: %)	R@1	R@1	R@1	R@1	R@5	R@1	R@5	R@1	R@5	R@1	R@5	R@1	R@5	R@1	R@5
CQAA	30.9	36.7	51.9	41.4	50.2	41.6	50.6	47.2	57.3	31.6	39.8	37.1	51.8	49.7	65.1
BuboQA	34.2	43.6	54.3	41.8	51.1	56.8	65.5	48.3	59.1	35.2	42.1	43.1	54.7	52.3	66.9
UBSQA	34.9	43.8	54.8	42.4	51.3	56.9	65.5	48.5	59.6	36.4	44.4	43.7	54.9	52.5	68.2
KGEQA	38.9	44.7	55.3	44.8	53.2	57.4	66.9	48.8	59.6	39.5	47.1	44.5	55.9	53.6	68.8
BERTQA	38.7	44.8	55.7	44.9	53.0	57.2	66.6	48.9	60.2	39.8	46.6	44.1	55.4	53.3	68.8
SAKE\S	37.8	42.2	53.2	43.2	51.9	56.2	65.3	48.5	57.8	38.3	45.9	43.5	54.2	52.4	67.7
CQAA+S	59.7	64.4	69.7	69.8	77.3	60.2	69.4	59.2	68.9	60.3	70.4	64.7	71.5	68.3	78.2
BuboQA+S	66.5	69.6	71.8	70.1	78.1	75.1	81.8	68.6	78.7	67.4	77.2	69.5	79.6	69.5	79.4
UBSQA+S	68.2	70.2	72.6	71.8	79.7	75.3	82.3	68.8	78.9	68.5	78.4	70.1	80.5	70.8	80.9
KGEQA+S	71.3	70.6	75.1	73.8	81.2	75.8	83.4	70.2	79.4	71.6	79.2	70.7	80.6	74.2	83.2
BERTQA+S	70.3	70.2	73.0	72.1	80.5	75.3	82.6	68.9	79.4	70.5	78.7	70.1	80.3	71.3	81.6
SAKE (ours)	**74.2**	**78.5**	**77.6**	**76.6**	**83.3**	**77.8**	**84.4**	**72.9**	**81.0**	**75.3**	**79.8**	**78.2**	**83.3**	**76.3**	**84.1**
SAKE\GloVe	60.3	67.7	59.8	67.6	74.3	74.4	79.9	56.5	64.0	59.1	62.6	61.3	66.4	57.4	65.1

relations of test data from the sub KG). SAKE and KGEQA+S are the top-2 methods in SQA processing as they use pre-trained KG embeddings to help recall missed facts followed by a joint similarity search. Moreover, SAKE outperforms KGEQA+S because it further uses more expressive GloVe embeddings to replace the entity/predicate mentions extracted from NLSQs to expand candidate facts.

Fourth, compared to the cross-validation results on the full datasets, SAKE's recall measures stay relatively stable on all human question datasets (labeled by ^{R} and ^{M}). The fluctuation of results on different datasets of the same city may be caused by the uneven sampling process in manual labeling. Nevertheless, the competitive results on human questions generated by two different methods show the effectiveness of our model driven by large-scale template-generated QA-pairs.

E. Effect of NLSQ Types

We scrutinize processing different NLSQ types on CPH^R, whose exemplified questions for a particular fact are shown in Table IV. Intuitively, nearby and location-name NLSQs whose query intents are implicitly related to user location or textual location name are more difficult to handle.

TABLE IV: Rephrased NLSQ Examples of a Spatial Fact in CPH^R

A fact: (`Sven_Risom`, *created*, `St._Anne's_Church,_Copenhagen`)

Basic: *Who builds the St. Anne's Church in Copenhagen?*
Class-based: *Which European people is the Sankt Annæ Kirke's creator?*
Nearby: *Which neighboring church is created by a Dane?* #with user location (55.6559, 12.6082)#
Location-name: *What building in København S district is built by Sven Risom?*

Table V reports $R@1$ measures for each NLSQ type. Compared to BERTQA, the best baseline without spatial fingerprints, SAKE achieves significant improvement on nearby and location-name NLSQs. Again, the result verifies that our proposed fingerprints are good at extracting spatial query intents from locations and text names. Moreover, SAKE always works better than or equally to KGEQA+S and BERTQA+S. That is because SAKE adopts embeddings full-stack to capture NLSQ semantics, a technique more robust than string matching in similarity measurement. In general, thanks to its deep integration of spatial and semantic similarities, SAKE holds clear advantages at answering NLSQs.

TABLE V: Result Effectiveness vs. NLSQ Types in CPH^R

NLSQ types	# of QA-pairs	R@1 (unit: %) SAKE	BERTQA	KGEQA+S	BERTQA+S
basic	22	**81.8**	77.3	**81.8**	**81.8**
class-based	369	**82.7**	82.1	82.1	82.4
nearby	361	**72.9**	30.7	64.5	59.3
location-name	277	**77.6**	57.0	74.0	74.4

F. Effectiveness of Spatial KG Embedding

We evaluate how well our spatial fingerprints capture the spatial closeness. In particular, we sample 200 GPS points inside a city's MBR and obtain its spatially-nearest entity[5] from the test data. Next, we encode each sampled GPS point as a spatial fingerprint and find its k nearest neighbors in the fingerprint space. The effectiveness is measured as $R@k$, the ratio of the nearest entity's spatial fingerprint being recalled in the kNN results. Two alternative fingerprint generation methods are compared to SAKE's. First, GPS2VEC [34] divides the city MBR as 1km×1km grids c_{ij} and computes the initial encoding for a GPS location $\mathbf{l}$ as $\{\exp(-|\mathbf{l}, c_{ij}.\mathbf{l}|_2/\sigma) \mid i, j = 1, 2, \ldots\}$, where σ is the attenuation coefficient and $c_{ij}.\mathbf{l}$ is the grid center. Such encodings replace SAKE's multi-scale encodings to learn fingerprints. Second, ISE-AGG does not learn fingerprints for ISEs but directly uses the mean of the neighboring ESEs' spatial fingerprints to represent an ISE. Varying the ratio of ISEs, we report $R@5$ measures in Fig. 8.

Spatial fingerprints generated by SAKE are the best to recall spatially-nearest entities in all datasets. When 10% entities

[5]We considered also ISEs and got back their GPS from the original KG.

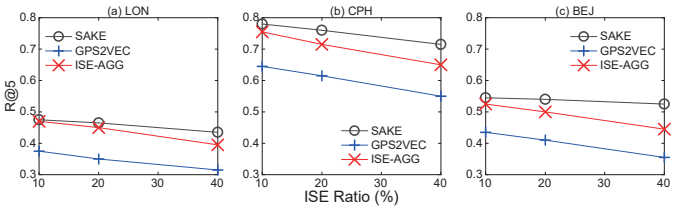


Fig. 8: Recall of Spatial Fingerprint vs. ISE Ratio

are evolved as ISEs, $R@5$ is as high as 0.78 in CPH and higher than 0.47 in LON. Considering LON contains over 13.6k entity candidates, this result is satisfactory[6]. GPS2VEC is always the worst as its encodings only represent locations relative to the falling grids, which is generally coarse to capture spatial features in different granularities. ISE-AGG's method of generating ISE fingerprints through the neighbors is inferior to SAKE, as it overlooks that two spatial entities connected in KG are likely to be spatially distant. Besides, as ISE ratio increases, less GPS information is preserved and all methods' recall deteriorates. However, SAKE decreases the slowest due to its ability to learn fingerprints for ISEs by modeling different KG relations between spatial entities. The difference in results on different datasets should be caused by the different numbers of candidate entities (see Section V-A).

Table VI shows a case study of SAKE's spatial fingerprints. We use an entity `Tiananmen`'s GPS location to search for its nearest neighbors in the fingerprint space. We make three key observations. First, the top-10 entities recalled are all within 2.5km of `Tiananmen`, and 7 of them are in the top-10 ground truth rank. Second, an ISE `Meridian_Gate` is successfully recalled even its geospatial attributes were erased before embedding. Third, some recalled entities are not close to `Tiananmen`, but they are closely related to `Tiananmen` in KG (i.e., `Xinhua_News_Agency`) or the administrative district that intersects `Tiananmen` (i.e., `Xuanwu_District,_Beijing`). The results verify that the fingerprints can recall spatially relevant entities, and restore the geospatial properties of ISEs through KG relations.

TABLE VI: Recall of `Tiananmen`'s GPS Point on BEJ

#	Entity Name	Distance (m)	True Rank
1	`Tiananmen`	0.0	1
2	`Forbidden_City_Concert_Hall`	558.9	7
3	`Zhongshan_Park_(Beijing)`	259.6	3
4	`Xinhua_News_Agency`	2402.2	90
5	`Gate_of_China_(Beijing)`	693.6	8
6	`Collections_of_the_Palace_Museum`	919.5	11
7	`Meridian_Gate`	574.5	7
8	`Tiananmen_Square`	447.5	6
9	`Xuanwu_District,_Beijing`	1630.3	45
10	`Beijing_Shejitan`	415.7	5

G. Effect of Candidate Fact Search

We design two alternatives to candidate fact search CFS (Algorithm 1). First, a brute force algorithm (BF) explores each fact category with k candidates and returns the category with the lowest overall scores. Second, a top sampling algorithm (TS) samples top-k' facts of each category and then finds top-k results only for the category with the lowest overall scores of top-k' facts. We compare the three algorithms in terms of search efficiency and result effectiveness. Varying k as 5, 25, 50, and 100, we report the average search time and hit@k of 500 NLSQs sampled from LON dataset.

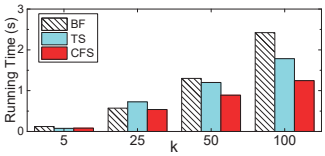


Fig. 9: Search Time on LON

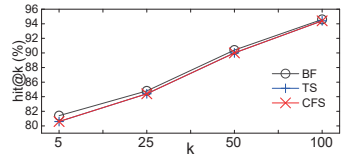


Fig. 10: hit@k on LON

Each algorithm's search time in Fig. 9 increases steadily with an increasing k. Among them, CFS is the fastest—it returns the top-100 results in around 1.3s on our largest dataset LON. Compared to BF, CFS and TS adopt top sampling to exclude unpromising categories. Their difference is that CFS's top sampling is integrated into the search and quickly comes down to finding best-matched categories without computing similarities for the sampled facts of each category. Therefore, the gap of TS and CFS expands when k increases.

Referring to Fig. 10, the hit@k measures of all algorithms are close. When $k = 5$, 403 out of all 500 NLSQs can get its closest answer included in CFS/TS results, while the number of BF is 407. In general, checking all categories does not make significant gains on recall but incurs more search costs. Moreover, we have tried larger k' to sample more facts in CFS/TS but do not see much improvement. The results verify that CFS with a small k' can achieve a good balance between search time and result effectiveness.

VI. Related Work

Question Answering over Knowledge Graph. KGQA techniques fall into four categories. (1) *Retrieval-based methods* extract discriminative features from the question and/or KG to retrieve candidate answers with high matching degrees. Yao et al. [33] identify the subgraph of entities from the question to match it to a KG pattern. This category highly relies on domain knowledge for feature engineering. (2) *Template-based methods* transform a question into an executable KG query through templates. Abujabal et al. [4] generate templates over QA-pairs and employ subquestion answering and stitching to handle complex questions. Hu et al. [15] propose a semantic query graph to interpret an NLQ and find answers through subgraph matching over KG. This category is susceptible to error propagation from template matching to query generation. (3) *Semantic parsing based methods* capture high-level paraphrase of a question to find answers with the highest paraphrase scores. Dong et al. [10] use CNNs to train the paraphrase scoring and answer selection end-to-end over QA-pairs. This category relies on large-scale training QA-pairs and can hardly deal with questions and answers out of the labeled data range. (4) *Embedding-based methods* project questions and candidate answers into latent space and then measure their

[6]After the candidate entities are recalled by the spatial fingerprint, SQA will further expand and rank the candidate facts and return the top ones. Therefore, a k much larger than 5 is likely used in practice, which leads to even better final QA results according to the $R@5$ results here.

matching scores. Also requiring sufficient training QA-pairs, this category, however, excels at synthesizing KG facts due to the intricate reasoning capabilities of embedding techniques.

A review of the embedding-based *simple* question answering methods including [7], [12], [16], [20], [23], [24] has been provided in [2]. Some embedding-based methods answer complex NLQs by finding KG subgraph as answers. Yang et al. [32] extract and embed lexical features (n-grams) from questions and logical features from facts to build their mapping. Dong et al. [11] decompose the embedding function as three CNN models, corresponding to the answer path, answer context, and answer types. Wang et al. [31] retrieve top-k answers to a query graph SG_Q by defining path semantic similarity over SG_Q's KG embeddings. There also exist works that utilize word and entity embeddings to map a fact to a sentence [8] or vice versa [9]. However, none of the aforementioned methods focus on interpreting spatial query intents in NLSQs.

Semantic-oriented Spatial Awareness. Ramalho et al. [26] propose a neural network-based encoder to capture the semantics of spatial relations in natural languages for generating scene images. Li et al. [17] propose a comprehension model to rewrite spatial-domain languages to relational database queries. Bateman et al. [5] design a linguistic ontology to enhance spatial natural language queries such as wayfinding and relationship inference. These studies apply to specific data formats or tasks different than KGQA. Focusing on physical relationships of POIs without any KG context, Mai et al. [22] study multi-scale location embeddings for place recognition.

Spatial KG Queries. Sun et al. [28] propose Riso-Tree that pre-materializes sub-graph entries on tree nodes to support spatial queries over KGs. Liagouris et al. [18] propose an encoding scheme for spatial RDF entities to support spatial filters and spatial join algorithms. Shi et al. [27] define top-k relevant semantic place retrieval to find spatial RDF entities containing the query keywords and being geographically close to the query location. These techniques focus on querying spatially and/or semantically close results from RDF stores but cannot handle NLQs. Wang et al. [30] design a formalized geographic KG model to answer geographic questions, however, in a SPARQL query form.

VII. Conclusion and Future Work

This work studies the spatial question answering over KG with embedding techniques. First, we learn the embeddings of KG entities/predicates in two processes for spatial and semantic properties, respectively. Subsequently, we map natural language questions to the learned embeddings of their mentioned entities and predicates. To answer spatial questions, we train a joint similarity function with question-answer pairs, and design a candidate search algorithm to find the spatially and semantically closest KG facts as the answers to spatial questions. The extensive experimental results verify the effectiveness and efficiency of our proposed framework.

For future work, it is interesting to integrate KG embedding and SQA processing procedures based on a multi-task learning paradigm. It is useful to extend the spatial KG embedding techniques to other mobility applications, such as location-based recommendation and POI semantics enrichment. Moreover, it is interesting to study more complex NLSQs and extend the proposal with advanced KG embedding and NLP techniques.

Acknowledgement. Lidan Shou's work was partly supported by the Key R&D Program of Zhejiang, China (No. 2021C009), the NSF-China (No. 62050099), and the NSF of Zhejiang, China (No. LY18F020005). The authors would like to thank Bang Van Bui and Xinyu Chen for preliminary experiments.

References

[1] https://nominatim.org/release-docs/develop/api/Overview/. (Accessed Oct 2020).
[2] https://github.com/sake-project/awesome-KBSQA/. (Accessed Jan 2021).
[3] https://github.com/huggingface/transformers. (Accessed Jan 2021).
[4] A. Abujabal, M. Yahya, M. Riedewald, and G. Weikum. Automated template generation for question answering over knowledge graphs. In *WWW*, pages 1191–1200, 2017.
[5] J. A. Bateman, J. Hois, R. Ross, and T. Tenbrink. A linguistic ontology of space for natural language processing. *Artificial Intelligence*, 174(14):1027–1071, 2010.
[6] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko. Translating embeddings for modeling multi-relational data. In *NIPS*, pages 2787–2795, 2013.
[7] A. Bordes, J. Weston, and N. Usunier. Open question answering with weakly supervised embedding models. In *ECML PKDD*, pages 165–180, 2014.
[8] BD. Trisedya, J. Qi, R. Zhang, and W. Wang. GTR-LSTM: A triple encoder for sentence generation from RDF data. In *ACL*, pages 1627–1637, 2018.
[9] BD. Trisedya, W. Gerhard, J. Qi, and R. Zhang. Neural relation extraction for knowledge base enrichment. In *ACL*, pages 229–240, 2019.
[10] L. Dong, J. Mallinson, S. Reddy, and M. Lapata. Learning to paraphrase for question answering. In *EMNLP*, pages 875–886, 2017.
[11] L. Dong, F. Wei, M. Zhou, and K. Xu. Question answering over freebase with multi-column convolutional neural networks. In *ACL*, pages 260–269, 2015.
[12] D. Golub and X. He. Character-level question answering with attention. In *EMNLP*, pages 1598–1607, 2016.
[13] X. Han, S. Cao, X. Lv, Y. Lin, Z. Liu, M. Sun, and J. Li. OpenKE: An open toolkit for knowledge embedding. In *EMNLP*, pages 139–144, 2018.
[14] S. He, K. Liu, G. Ji, and J. Zhao. Learning to represent knowledge graphs with Gaussian embedding. In *CIKM*, pages 623–632, 2015.
[15] S. Hu, L. Zou, J X. Yu, H. Wang, and D. Zhao. Answering Natural Language Questions by Subgraph Matching over Knowledge Graphs. In *ICDE*, pages 1815-1816, 2018.
[16] X. Huang, J. Zhang, D. Li, and P. Li. Knowledge graph embedding based question answering. In *WSDM*, pages 105–113, 2019.
[17] J. Li, W. Wang, W.-S. Ku, Y. Tian, and H. Wang. SpatialNLI: A spatial domain natural language interface to databases using spatial comprehension. In *SIGSPATIAL*, pages 339–348, 2019.
[18] J. Liagouris, N. Mamoulis, P. Bouros, and M. Terrovitis. An effective encoding scheme for spatial RDF data. *PVLDB*, 7(12):1271–1282, 2014.
[19] Y. Lin, Z. Liu, M. Sun, Y. Liu, and X. Zhu. Learning entity and relation embeddings for knowledge graph completion. In *AAAI*, pages 2181–2187, 2015.
[20] D. Lukovnikov, A. Fischer, and J. Lehmann. Pretrained transformers for simple question answering over knowledge graphs. In *ISWC*, pages 470–486, 2019.
[21] F. Mahdisoltani, J. Biega, and F. M. Suchanek. YAGO3: A knowledge base from multilingual wikipedias. In *CIDR*, 11 pages, 2013.
[22] G. Mai, K. Janowicz, B. Yan, R. Zhu, L. Cai, and N. Lao. Multi-scale representation learning for spatial feature distributions using grid cells. In *ICLR*, 15 pages, 2020.
[23] S. Mohammed, P. Shi, and J. Lin. Strong baselines for simple question answering over knowledge graphs with and without neural networks. In *NAACL*, pages 291–296, 2018.
[24] M. Petrochuk and L. Zettlemoyer. Simplequestions nearly solved: a new upper-bound and baseline approach. In *EMNLP*, pages 554–558, 2018.
[25] J. Pennington, R. Socher, and C. D. Manning. Glove: Global vectors for word representation. In *EMNLP*, pages 1532–1543, 2014.
[26] T. Ramalho, T. Kočiskỳ, F. Besse, et al. Encoding spatial relations from natural language. *arXiv:1807.01670*, 2018.
[27] J. Shi, D. Wu, and N. Mamoulis. Top-k relevant semantic place retrieval on spatial RDF data. In *SIGMOD*, pages 1977–1990, 2016.
[28] Y. Sun and M. Sarwat. A generic database indexing framework for large-scale geographic knowledge graphs. In *SIGSPATIAL*, pages 289–298, 2018.
[29] D. Vrandečić and M. Krötzsch. Wikidata: a free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014.
[30] S. Wang, X. Zhang, P. Ye, M. Du, Y. Lu, and H. Xue. Geographic knowledge graph (GeoKG): A formalized geographic knowledge representation. *ISPRS International Journal of Geo-Information*, 8(4):184, 2019.
[31] Y. Wang, A. Khan, T. Wu, J. Jin, and H. Yan. Semantic guided and response times bounded top-k similarity search over knowledge graphs. In *ICDE*, pages 445–456, 2020.
[32] M.-C. Yang, D.-G. Lee, S.-Y. Park, and H.-C. Rim. Knowledge-based question answering using the semantic embedding space. *Expert Systems with Applications*, 42(23):9086–9104, 2015.
[33] X. Yao and B. Van Durme. Information extraction over structured data: Question answering with freebase. In *ACL*, pages 956–966, 2014.
[34] Y. Yin, Z. Liu, Y. Zhang, S. Wang, R R. Shah, and R. Zimmermann Durme. GPS2Vec: Towards Generating Worldwide GPS Embeddings. In *SIGSPATIAL*, pages 416–419, 2019.